

Unidad 1: Arquitectura y Diseño.

Tema 1. Arquitectura. Diseño arquitectónico. Decisiones de diseño arquitectónico. Organización del sistema. Estilos de descomposición modular. Estilos de control. Arquitecturas de referencia. Arquitecturas de sistemas distribuidos. Arquitecturas de aplicaciones

Arquitectura del Sistema.

Afecta al rendimiento y mantenibilidad. Las decisiones de diseño se basan en requerimientos no funcionales como rendimiento, seguridad y mantenibilidad.

Decisiones de Diseño Arquitectónico.

Involucran responder a preguntas clave como la existencia de una arquitectura genérica aplicable, la distribución del sistema entre procesadores, y la selección de estilos arquitectónicos y estrategias para estructurar el sistema.

El diseño arquitectónico culmina en un documento que incluye representaciones gráficas y texto descriptivo del sistema.

Se reconoce que, aunque cada sistema de software es único, aquellos dentro del mismo dominio de aplicación a menudo comparten arquitecturas similares que reflejan los conceptos fundamentales de dicho dominio. Es común que los sistemas grandes no utilicen un solo estilo arquitectónico; diferentes partes del sistema pueden diseñarse utilizando estilos arquitectónicos diversos. En algunos casos, la arquitectura completa del sistema puede ser una arquitectura compuesta, creada mediante la combinación de diferentes estilos arquitectónicos.

Rendimiento.

Si el rendimiento es un requerimiento crítico, la arquitectura debería diseñarse para identificar las operaciones críticas en un pequeño número de subsistemas, con tan poca comunicación como sea posible entre estos subsistemas.

Protección.

Si la protección es un requerimiento crítico, debería usarse una arquitectura estructurada en capas, con los recursos más críticos protegidos en las capas más internas y aplicando una validación de seguridad de alto nivel en dichas capas.

Seguridad.

La arquitectura debería diseñarse para que las operaciones relacionadas con la seguridad se localizaran en un único subsistema o en un pequeño número de subsistemas.

Disponibilidad.

La arquitectura debería diseñarse para incluir componentes redundantes y para que sea posible reemplazar y actualizar componentes sin detener el sistema.

Mantenibilidad.

La arquitectura del sistema debería diseñarse usando componentes independientes de grano fino que puedan modificarse con facilidad.

Diagrama de Bloques: Consideraciones.

Bass (Bass et al., 2003) argumenta que los diagramas simples de cajas y líneas carecen de utilidad como representaciones arquitectónicas porque no revelan la naturaleza de las relaciones entre los componentes del sistema ni las propiedades visibles externamente de dichos componentes. A pesar de ello, estos modelos son efectivos para la comunicación con los stakeholders y la planificación del proyecto, ya que omiten detalles y se centran en la visualización general.

Modelo-Estructural.

Muestra los subsistemas o componentes como unidades separadas, revelando la organización estática del sistema.

Modelo de Proceso Dinámico.

Ilustra cómo se organiza el sistema en procesos durante la ejecución, pudiendo diferir del modelo estructural.

Modelo de Interfaz.

Define los servicios que cada subsistema ofrece a través de su interfaz pública, facilitando la interacción entre componentes.

Modelo de Relaciones.

Muestra cómo los subsistemas se relacionan entre sí, como el flujo de datos. Es fundamental para entender cómo interactúan las diferentes partes del sistema y cómo se transfiere la información.

Modelo Distribuido.

Describe la distribución de los subsistemas a través de múltiples computadoras. Es esencial para sistemas que operan en una red y necesitan manejar componentes redundantes y actualizaciones sin detener el sistema.

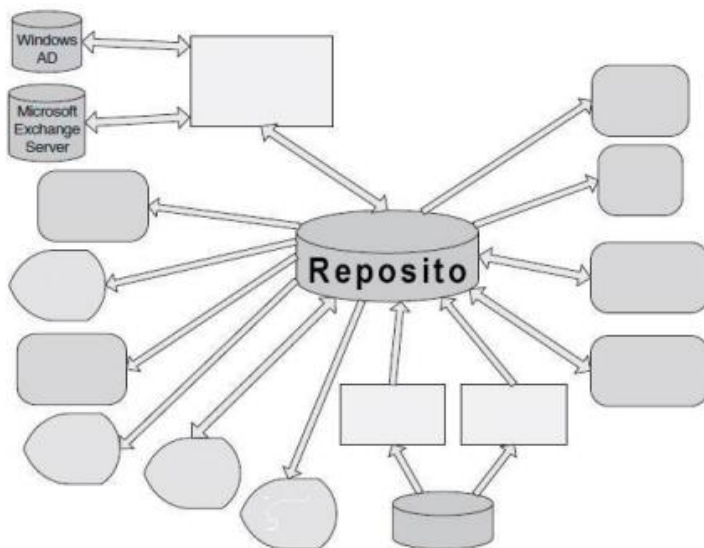
Organización del Sistema.

Refleja la estrategia fundamental utilizada para estructurarlo. Se toman decisiones sobre el modelo organizacional al principio del proceso de diseño arquitectónico. A menudo, el modelo de subsistemas contiene más detalle que el organizacional. Los tres estilos organizacionales ampliamente utilizados son: el estilo de repositorio de datos, el estilo de servicios y servidores compartidos, y el estilo por capas, donde el sistema se organiza en un conjunto de capas funcionales.

Repositorio (Shared-Data).

- Contexto: Componentes que necesitan compartir grandes cantidades de datos.
- Problema: Almacenar y manipular datos persistentes accesibles por múltiples componentes.
- Solución: Intercambio de datos persistentes entre accesorios y un almacén de datos compartido.
- Elementos: Almacén central de datos; componentes independientes que acceden, modifican o publican datos en ese repositorio.
- Debilidades:

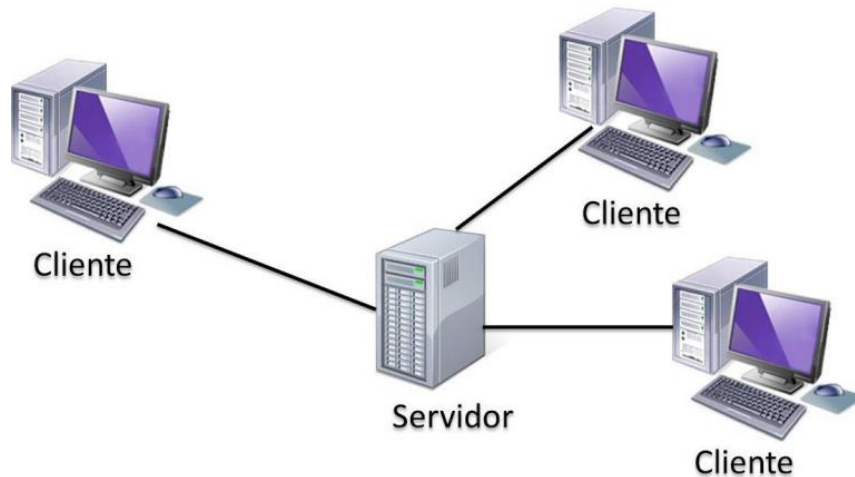
- Alta dependencia del repositorio como punto único de fallo.
- Riesgos de bloqueo de acceso y problemas de consistencia.
- Puede limitar la escalabilidad.
- Ventajas:
 - Facilita la integración y el intercambio de información.
 - Promueve la reutilización.
 - Simplifica el mantenimiento de datos persistentes.
- Gráficamente:



Cliente-Servidor.

- Contexto: Acceso a recursos y servicios compartidos por múltiples clientes.
- Problema: Administrar recursos compartidos para mejorar la modificabilidad y reutilización.
- Solución: Interacción cliente-servidor con un servidor que proporciona servicios a los clientes.
- Elementos: Cliente, Servidor, Conector de solicitud/respuesta (protocolo como HTTP o TCP/IP)
- Debilidades:
 - El servidor puede ser cuello de botella y un único punto de falla.
 - Dificultad para reubicar lógica una vez implementada.

- Ventajas:
 - Claramente estructurado y ampliamente utilizado.
 - Favorece la reutilización de servicios.
 - Escalable al permitir múltiples clientes y réplicas de servidor.
- Gráficamente:



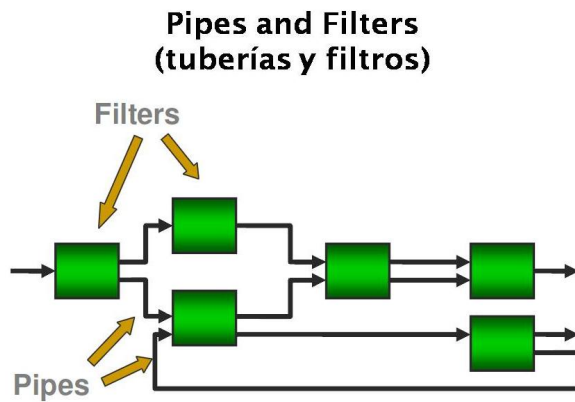
Arquitectura en Capas.

- Contexto: Sistemas complejos que requieren desarrollo independiente.
- Problema: Necesidad de segmentación para soportar portabilidad y reutilización.
- Solución: División en capas con restricciones unidireccionales de uso.
- Elementos: Capas (módulos) con relaciones de uso unidireccionales
- Debilidades:
 - Añade complejidad y penalización en el rendimiento.
 - Dificultades si no se diseña correctamente (puede obstaculizar en lugar de ayudar).
- Ventajas:
 - Buena separación de responsabilidades.
 - Favorece la portabilidad y mantenibilidad.
 - Facilita la evolución modular del sistema
- Gráficamente:

Capa de presentación
Capa de lógica de negocios
Capa de persistencia de datos
Capa de base de datos.

Tuberías y Filtros (Pipes & Filters).

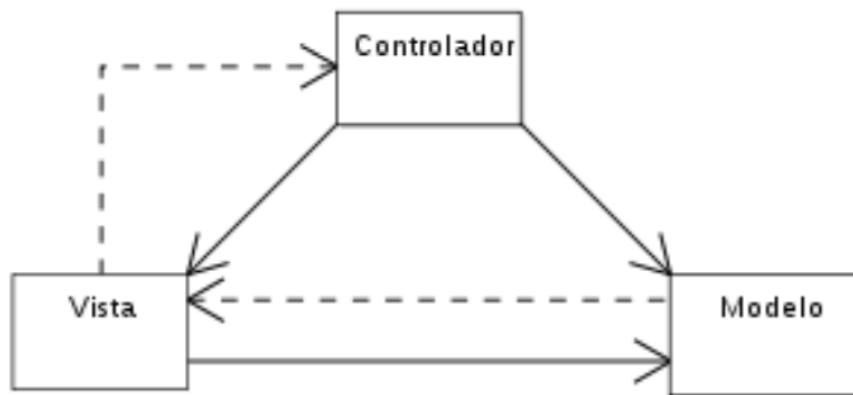
- Contexto: Sistemas que transforman flujos de datos discretos.
- Problema: Necesidad de componentes reutilizables con interacciones simples.
- Solución: Filtros que transforman datos y tuberías que los transportan.
- Elementos: Filtros (procesan datos), Tuberías (conectores entre filtros).
- Debilidades:
 - No adecuado para sistemas interactivos.
 - Sobrecarga computacional por múltiples hilos/procesos.
 - Riesgo alto si falla un componente.
- Ventajas:
 - Reutilizables y fácilmente componibles.
 - Alta concurrencia y paralelismo.
 - Facilita el razonamiento sobre el comportamiento global.
- Gráficamente:



Modelo-Vista-Controlador (MVC).

- Contexto: Software de interfaz de usuario sujeto a modificaciones frecuentes.
- Problema: Separación de la funcionalidad de la interfaz de usuario del resto del sistema.
- Solución: División en modelo, vista y controlador para gestionar cambios.
- Elementos: Modelo (datos y lógica), Vista (UI), Controlador (gestiona la interacción) .
- Debilidades:
 - Complejidad innecesaria para interfaces simples.
 - No siempre encaja bien con toolkits de UI comerciales.
- Ventajas:
 - Alta cohesión y bajo acoplamiento.
 - Facilita el mantenimiento y las pruebas independientes.
 - Soporta múltiples vistas y controladores simultáneamente.

- Gráficamente:



Estilos de Descomposición Modular.

En los estilos de descomposición modular, no hay una distinción clara entre subsistemas y módulos, pero es útil pensar en ellos de la siguiente manera: un subsistema es un sistema en sí mismo, mientras que un módulo es un componente que proporciona servicios a otros módulos dentro del subsistema. Los subsistemas se componen de módulos y tienen interfaces definidas para comunicarse con otros subsistemas. Hay dos estrategias principales para descomponer un subsistema en módulos: la descomposición orientada a objetos y la descomposición orientada a flujos de funciones. En ambos casos, los módulos pueden implementarse como componentes secuenciales o como procesos.

Descomposición Orientada a Objetos.

- Concepto: Se descompone un sistema en un conjunto de objetos que se comunican entre sí.
- Características: Cada objeto tiene un estado privado y operaciones definidas sobre ese estado.
- Uso: Adecuado para sistemas donde los datos y su manipulación están estrechamente relacionados.

Descomposición Orientada a Flujos de Funciones.

- Concepto: Se descompone un sistema en módulos funcionales que aceptan datos de entrada y los transforman en datos de salida.

- Características: Los módulos son transformaciones funcionales y pueden implementarse como componentes secuenciales o procesos.
- Uso: Apropiado para sistemas donde el procesamiento de datos es la principal preocupación y se puede realizar de manera secuencial o en lotes.

Estilos de Control.

Relacionados con el flujo de control entre subsistemas, pueden ser centralizados, donde un subsistema controla a los demás, o basados en eventos, donde los subsistemas responden a eventos generados externamente. Estos estilos de control se utilizan junto con estilos estructurales, los cuales pueden implementarse utilizando control centralizado o control basado en eventos.

Control centralizado.

Un subsistema tiene toda la responsabilidad para controlar e iniciar y detener a otros subsistemas. También puede devolver el control a otro subsistema, pero esperará que le sea devuelta la responsabilidad del control.

- **El modelo de llamada-retorno.**

Es el modelo usual de subrutina descendente en donde el control comienza al inicio de una jerarquía de subrutinas y, a través de las llamadas a subrutinas. El control pasa a niveles inferiores en el árbol de la jerarquía. El modelo de subrutinas solamente es aplicable a sistemas secuenciales.

- **El modelo del gestor.**

Este es aplicable a sistemas concurrentes. Un componente del sistema se diseña como un gestor del sistema y controla el inicio, parada y coordinación del resto de los procesos del sistema. Un proceso es un subsistema o módulo que puede ejecutarse en paralelo con otros procesos. Una variante de este modelo también puede aplicarse a sistemas secuenciales en los que la rutina de gestión llama a subsistemas particulares dependiendo de los valores de algunas variables de estado. Esto normalmente se implementa como una sentencia case.

Control basado en eventos.

En lugar de que la información de control esté embebida en un subsistema, cada subsistema puede responder a eventos generados externamente. Estos eventos podrían provenir de otros subsistemas o del entorno del sistema.

Se diferencia de los modelos de control centralizados en que las decisiones de control se basan en eventos generados externamente en lugar de en variables de estado internas del sistema. En este enfoque, el sistema responde a eventos externos desencadenando acciones específicas, lo que permite una mayor flexibilidad y capacidad de respuesta a cambios dinámicos en el entorno del sistema.

Modelos de transmisión (broadcast).

En estos modelos, un evento se transmite a todos los subsistemas. Cualquier subsistema que haya sido programado para manejar ese evento puede responder a él.

Modelos dirigidos por interrupciones.

Éstos se usan exclusivamente en sistemas de tiempo real, en donde las interrupciones externas son detectadas por un manejador de interrupciones. A continuación, éstas se envían a algún otro componente para su procesamiento.

Arquitecturas de referencia.

Hay dos tipos de modelos arquitectónicos de dominio específico:

Modelos genéricos.

Son abstracciones obtenidas a partir de varios sistemas reales. Encapsulan las características principales de estos sistemas. Por ejemplo, en sistemas de tiempo real, podría haber modelos arquitectónicos genéricos de diferentes tipos de sistemas tales como sistemas de recolección de datos o sistemas de monitorización.

Modelos de referencia.

Son más abstractos y describen una clase más amplia de sistemas. Constituyen un modo de informar a los diseñadores sobre la estructura general de esta clase de

sistemas. Los modelos de referencia normalmente se obtienen a partir de un estudio del dominio de la aplicación. Representan una arquitectura ideal que incluye todas las características que los sistemas podrían incorporar.

Arquitecturas de Aplicaciones.

Se refieren a la estructura general de diferentes tipos de sistemas de software, como sistemas de procesamiento de datos, transacciones, eventos y lenguajes. Estas arquitecturas reflejan las operaciones y el flujo de datos dentro de las aplicaciones y cómo se configuran para cumplir con los requisitos específicos del negocio.

A continuación, se describen las arquitecturas de 4 grandes tipos de aplicaciones:

Aplicaciones de Procesamiento de Datos.

Operan en modo diferido, generalmente con una estructura de entrada-proceso salida. Procesan registros o transacciones en serie, sin mantener estado entre ellas, y son comunes en operaciones administrativas como nóminas y facturación.

Aplicaciones de Procesamiento de Transacciones.

Son interactivos y centrados en bases de datos, procesan peticiones de usuarios para obtener o actualizar información. Diseñados para que las acciones de los usuarios no interfieran entre sí y mantengan la integridad de la base de datos.

Aplicaciones de Procesamiento de Eventos.

Responden a eventos impredecibles en el entorno del sistema o interfaz de usuario. Incluyen sistemas de tiempo real que realizan acciones en respuesta a estímulos externos y sistemas de edición que interpretan eventos de la interfaz de usuario.

Aplicaciones de Procesamiento de Lenguajes.

Aceptan lenguaje natural o artificial como entrada y generan otra representación como salida. Incluyen compiladores que traducen lenguajes de programación a código máquina y otros sistemas que traducen descripciones de datos a comandos de consulta.

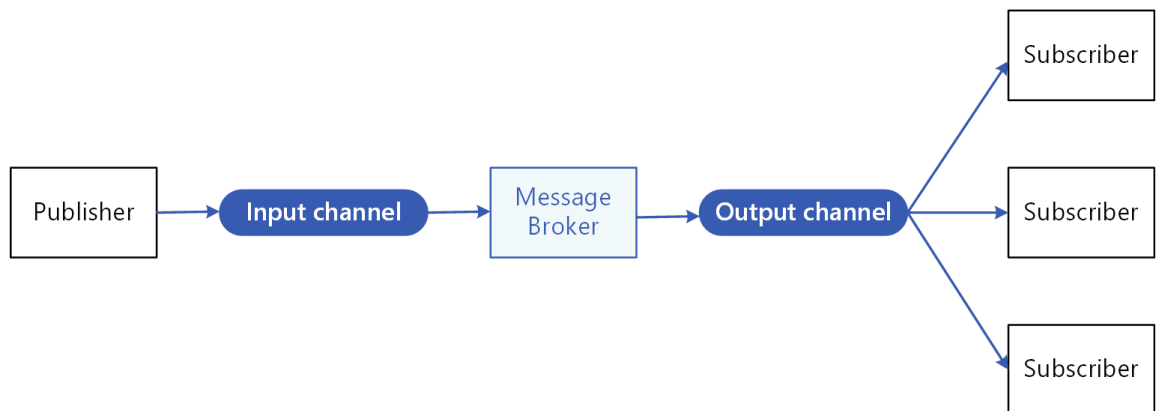
Arquitecturas de Sistemas Distribuidos.

Estas arquitecturas se centran en cómo se distribuyen los subsistemas entre las computadoras y cómo se organizan para trabajar juntos de manera eficiente. Los sistemas distribuidos permiten el uso efectivo de múltiples procesadores conectados en red, facilitando la adición o actualización de servidores de manera transparente.

Publicador-Suscriptor (Publish-Subscribe).

- Contexto: Interacción entre múltiples productores y consumidores de datos independientes.
- Problema: Transmitir mensajes sin que las partes estén conscientes de la identidad o existencia de las demás.
- Solución: Los componentes interactúan a través de mensajes anunciados o eventos, utilizando un bus de eventos para la entrega.
- Elementos: Publicadores, Suscriptores, Canal de eventos (bus o middleware).
- Debilidades:
 - Difícil de depurar y probar.
 - Sin garantías de orden ni entrega en muchos casos.
 - Falta de conocimiento entre emisores y receptores puede dificultar la trazabilidad.
- Ventajas:
 - Bajo acoplamiento.
 - Escalabilidad y flexibilidad.
 - Alta capacidad de adaptación a cambios dinámicos.

- Gráficamente:



Arquitectura Orientada a Servicios (SOAP).

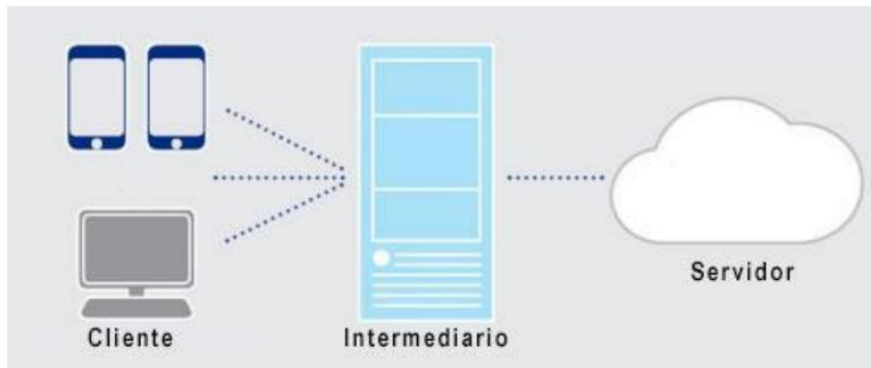
- Contexto: Proveedores y consumidores de servicios que requieren interoperabilidad.
- Problema: Integrar componentes distribuidos de diferentes plataformas y organizaciones.
- Solución: Colección de componentes que proveen/consumen servicios a través de conectores.
- Elementos: Servicios, Conectores (generalmente usando WSDL, SOAP, UDDI).
- Debilidades:
 - Sobrecarga por estandarización.
 - Complejidad en la gestión de contratos de servicio y orquestación.
- Ventajas:
 - Interoperabilidad entre plataformas.
 - Reutilización de servicios.
 - Adaptable a entornos empresariales distribuidos.
- Gráficamente:



Intermediario (Broker).

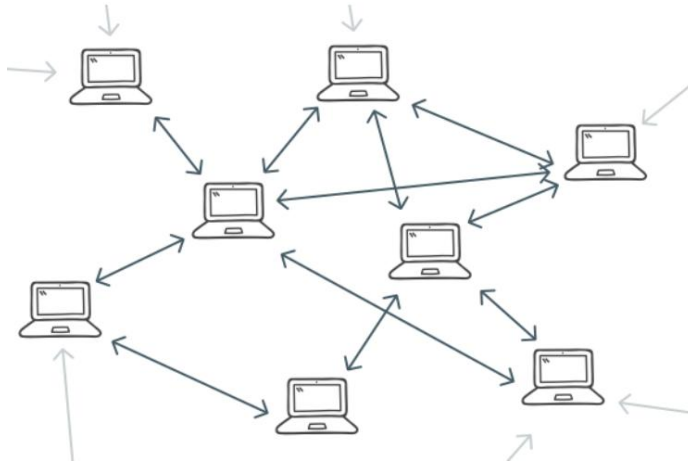
- Contexto: Sistemas con servicios distribuidos en múltiples servidores.
- Problema: Estructuración para ocultar la naturaleza y ubicación de los proveedores.
- Solución: Uso de intermediarios para conectar clientes y servidores.
- Elementos: Cliente, Servidor, Broker, Proxies en cliente y servidor.
- Debilidades:
 - Añade latencia y complejidad.
 - Punto único de falla si no se diseña adecuadamente.
 - Dificultades en testeo y depuración.
- Ventajas:
 - Oculta detalles de ubicación del servidor.
 - Mejora la disponibilidad.
 - Balancea carga entre servidores disponibles.

- Gráficamente:



De igual a igual (Peer-to-Peer | P2P).

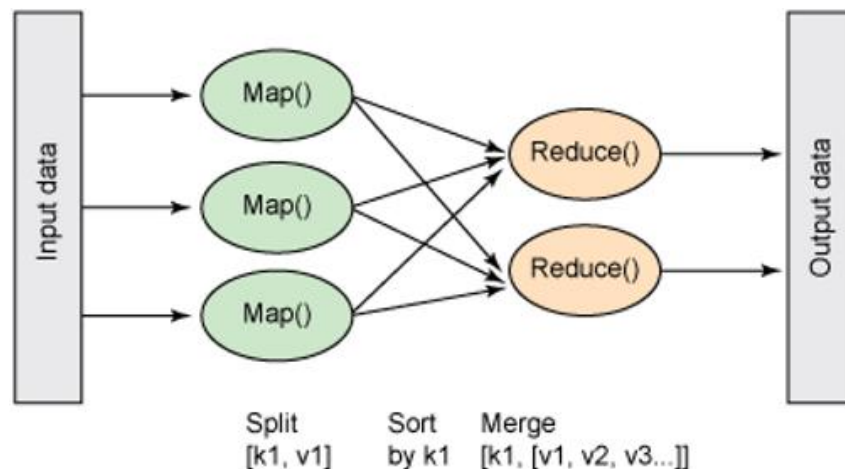
- Contexto: Entidades distribuidas que cooperan para brindar un servicio comunitario.
- Problema: Conectar entidades “iguales” para compartir servicios con alta disponibilidad.
- Solución: Componentes que interactúan como iguales, sin dependencia central.
- Elementos: Peers (todos actúan como clientes y servidores), protocolo común, red distribuida
- Debilidades:
 - Difícil de coordinar y controlar.
 - Riesgos de seguridad y replicación ineficiente.
- Ventajas:
 - Alta disponibilidad.
 - Buena escalabilidad.
 - Sin dependencia de un nodo central.
- Gráficamente:



Map-Reduce.

- Contexto: Necesidad de analizar rápidamente enormes volúmenes de datos.
- Problema: Realizar de manera eficiente y paralela la agrupación de un gran conjunto de datos.
- Solución: Infraestructura especializada que asigna software a nodos en un entorno paralelo y maneja la clasificación de datos.
- Elementos: Funciones Map (transforman), Reduce (agregan), Infraestructura para orquestación y tolerancia a fallos
- Debilidades:
 - No justificable para conjuntos de datos pequeños.
 - Difícil de dividir en partes equivalentes.
 - Orquestación compleja si se requiere múltiples reducciones.
- Ventajas:
 - Altamente escalable.
 - Procesamiento distribuido eficiente.
 - Buena tolerancia a fallos.

- Gráficamente:

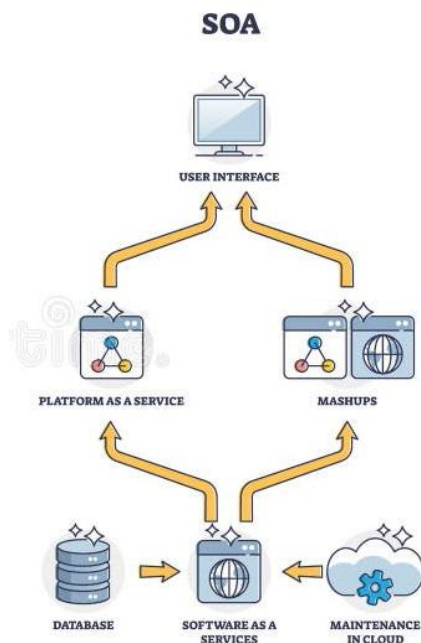


Arquitectura Orientada a Machine Learning.

- Contexto: Evolución natural desde arquitecturas monolíticas y orientadas a servicios, adaptadas al auge de los sistemas que aprenden de los datos.
- Problema: Complejidad en el tratamiento de datos, mantenimiento del rendimiento del modelo ante cambios, y necesidad de monitoreo constante.
- Solución: Diseñar arquitecturas con componentes específicos de ML (entrenamiento, inferencia, monitoreo), integrando equipos multidisciplinarios y poniendo foco en la gestión y calidad de los datos.
- Elementos: Data pipeline, reentrenamiento, monitoreo. Componentes como Feature Store, Orquestación, Evaluación, Interacción, Predicción, etc.
- Debilidades:
 - Alta sensibilidad a los datos.
 - Complejidad en la implementación y mantenimiento.
 - Riesgos éticos y de falta de explicabilidad del modelo.
- Ventajas:
 - Adaptabilidad del sistema a nuevos datos.
 - Aprendizaje automático continuo.
 - Capacidad de mejorar decisiones complejas y procesar grandes volúmenes de datos.
- Gráficamente:

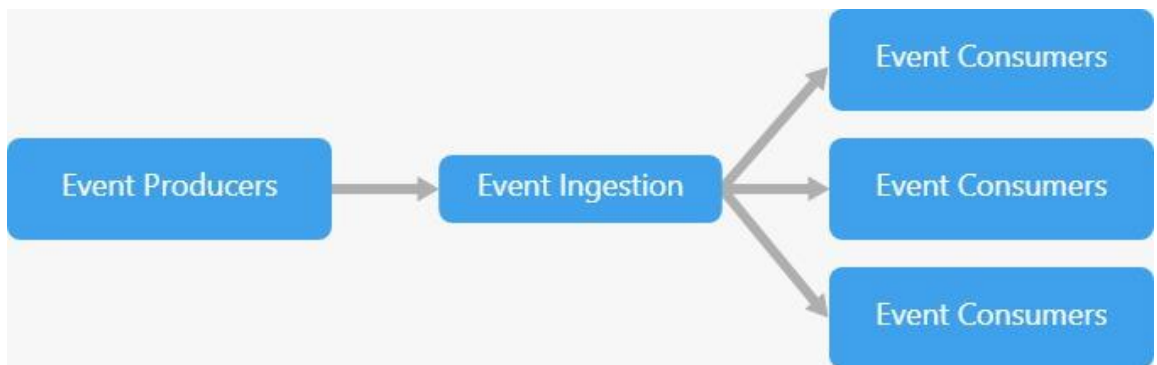
Arquitectura Orientada a Servicios (SOA).

- Contexto: Nació para dividir grandes sistemas en servicios independientes y reutilizables, tras el agotamiento de los monolitos.
- Problema: Necesidad de integrar sistemas diversos y adaptarse rápidamente a los cambios del negocio.
- Solución: Uso de servicios autónomos que interactúan mediante protocolos estándar (HTTP, SOAP, REST).
- Elementos: Servicios publicados en registros, middleware, consumidores, interfaces de red.
- Debilidades:
 - Complejidad en la gestión de servicios.
 - Costos iniciales y sobrecarga en el rendimiento.
- Ventajas:
 - Modularidad, escalabilidad, y facilidad de mantenimiento.
 - Permite la reutilización y adaptación a nuevas necesidades.
 - Integración sencilla de sistemas antiguos.
- Gráficamente:



Arquitectura Orientada a Eventos.

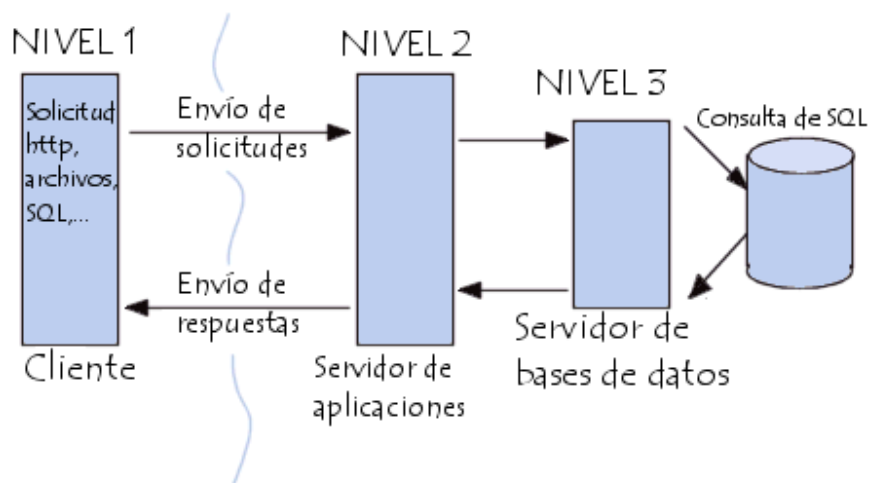
- Contexto: Creada para aplicaciones que requieren procesamiento en tiempo real y baja dependencia entre componentes.
- Problema: Dificultad para responder a eventos en tiempo real en sistemas acoplados.
- Solución: Uso de eventos como mecanismo de comunicación entre servicios/productores y consumidores mediante brokers.
- Elementos: Productores, consumidores, eventos, brokers, canales de comunicación.
- Debilidades:
 - Complejidad en el manejo de errores.
 - Latencia y dificultad para rastrear eventos.
- Ventajas:
 - Bajo acoplamiento y alta escalabilidad.
 - Flexibilidad y adaptabilidad a cambios.
 - Procesamiento eficiente y en tiempo real.
- Gráficamente:



Arquitectura Multinivel (Multi-Tier).

- Contexto: Surge como evolución del modelo cliente-servidor, para mejorar la organización y escalabilidad de aplicaciones.
- Problema: Mantenimiento complejo y poca escalabilidad en sistemas monolíticos o de dos capas.

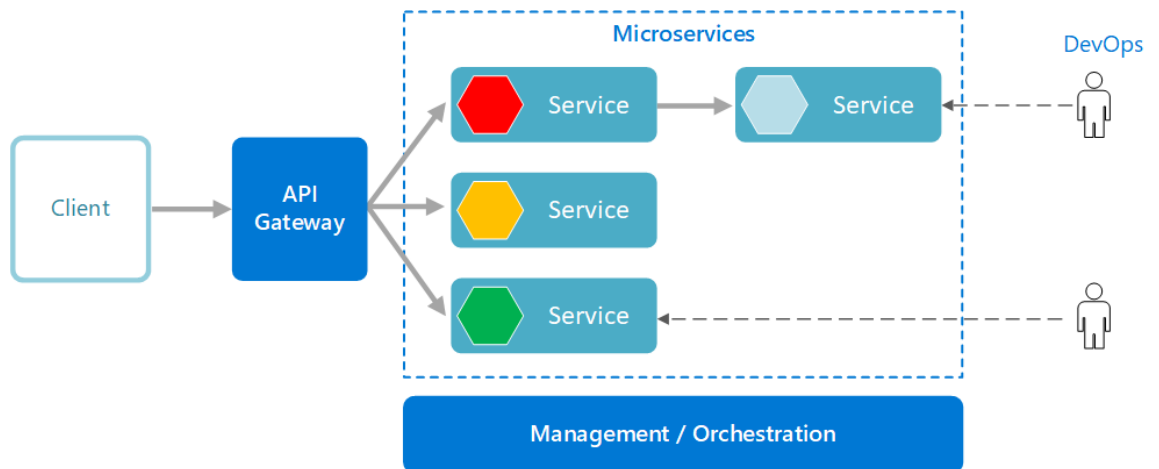
- Solución: Dividir el sistema en niveles: presentación (cliente), lógica de negocio (aplicación) y datos (base de datos).
- Elementos: Interfaz de usuario, lógica de negocio, acceso a datos (puede incluir más capas como seguridad o servicios web).
- Debilidades:
 - Latencia entre capas.
 - Infraestructura costosa.
 - Complejidad en el desarrollo y mantenimiento.
- Ventajas:
 - Mejor escalabilidad, seguridad y modularidad.
 - Facilita el mantenimiento y la reutilización.
 - Separación clara de responsabilidades.
- Gráficamente:



Arquitectura de Microservicios

- Contexto: Evolución de SOA, promovida por necesidades de agilidad, escalabilidad y despliegues independientes.
- Problema: Limitaciones de escalabilidad, mantenimiento y despliegue en sistemas monolíticos.
- Solución: Descomponer una aplicación en servicios pequeños e independientes que se comunican entre sí por APIs.

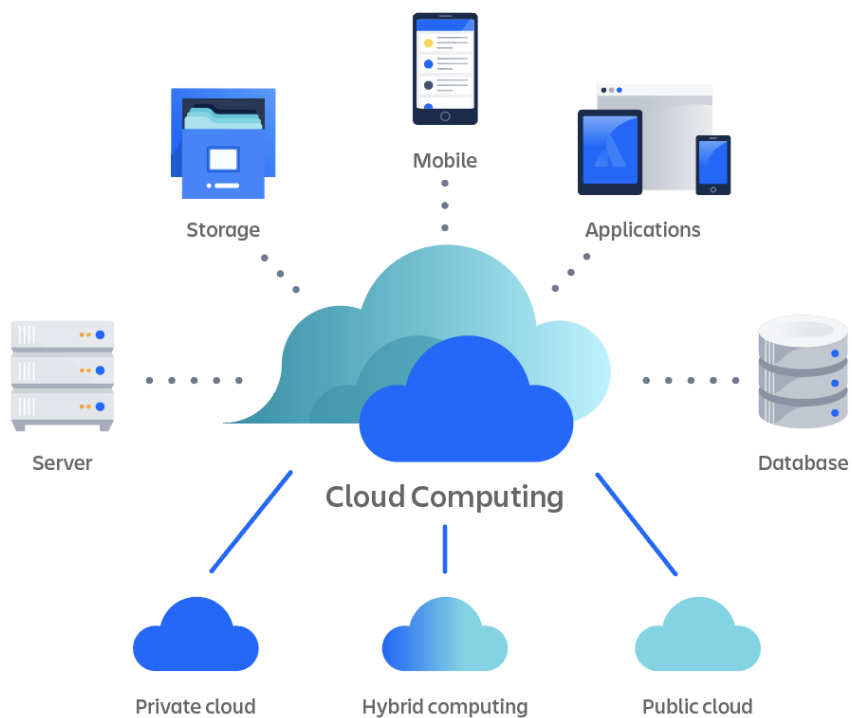
- Elementos: API Gateway, Balanceador de carga, Base de datos por servicio, Contenerización (Docker), Orquestación (Kubernetes), CI/CD, Mensajería (Kafka, RabbitMQ).
- Debilidades:
 - Complejidad en la comunicación y monitoreo.
 - Difícil mantener la consistencia de datos.
 - Seguridad y pruebas distribuidas complejas.
- Ventajas:
 - Escalabilidad independiente por servicio.
 - Despliegues rápidos y seguros.
 - Alta resiliencia ante fallos y flexibilidad tecnológica.
- Gráficamente:



Cloud Computing

- Contexto: Modelo moderno que sustituye la infraestructura física local por servicios en la nube (ej. AWS, Azure, GCP).
- Problema: Altos costos de infraestructura física, falta de escalabilidad y mantenimiento complejo.
- Solución: Acceder a recursos tecnológicos (almacenamiento, redes, servidores, software) a través de internet en modalidad “pago por uso”.
- Elementos: IaaS, PaaS, SaaS; Nube pública, privada e híbrida. Centros de datos, virtualización, red, gestión remota.

- Debilidades:
 - Dependencia de conexión a internet.
 - Riesgos de seguridad y privacidad de datos.
 - Pérdida de control sobre la infraestructura.
- Ventajas:
 - Escalabilidad inmediata.
 - Accesibilidad global.
 - Ahorro en costos de hardware y mantenimiento.
- Gráficamente:



Tema 2: Diseño de Software. Diseño de Interfaz de Usuario.

Del Análisis al Diseño. Conceptos Principales del Diseño. Diseño de Interfaz de Usuario. Diseño de Componentes. Independencia Funcional. El procedimiento de derivación. Análisis de transacciones y transformación. Evaluación y refinamiento: criterios, acoplamiento y cohesión. Documentación del Diseño. Diseño Ux/ Ui. Técnicas y consideraciones.

Diseño de Arquitectura de Módulos:

Objetivos.

- Diseñar la estructura modular de los procesos de cada subsistema.
- Partir del modelo de procesos de ASI y catálogo de requisitos.
- Realizar diseño detallado de la interfaz de usuario.

El Proceso de Diseño.

- Traducir requisitos en una representación física del software.
- Incluye diseño de datos, arquitectónico, de interfaz y procedimental.

Definición de Arquitectura del Sistema.

- Establecer la arquitectura general del sistema.
- Incluir particiones físicas y descomposición lógica en subsistemas.

Estrategias de Diseño.

- Aplicar principios como abstracción, modularidad y encapsulamiento.
- Refinar el diseño para mejorar la cohesión y reducir el acoplamiento.

Diseño Físico de Datos:

Objetivo.

Definir la estructura física de datos del sistema a partir del modelo lógico o de clases.

Proceso.

Incluye el “paso a tablas” para SGBDR y la optimización de accesos a datos para mejorar tiempos de respuesta.

Verificación y Aceptación de la Arquitectura.

- Objetivos: Asegurar la calidad técnica de los modelos de diseño y su viabilidad.
- Proceso: Incluye la verificación de cada modelo y la coherencia entre ellos, así como la aceptación del diseño por parte de los encargados.

Generación de Especificaciones de Construcción.

- Objetivo: Crear especificaciones detalladas para la construcción del sistema.
- Incluye: Especificación del entorno de construcción, descripción de componentes y especificación de la estructura física de datos.

Especificación Técnica del Plan de Pruebas.

- Objetivo: Detallar el plan de pruebas del sistema en varios niveles.
- Incluye: Pruebas unitarias, de integración, de implantación y de aceptación, así como la definición de casos de prueba.

El Proceso de Diseño:

Definición y Objetivo.

El diseño es un proceso iterativo para traducir requisitos en una representación física del software, con el objetivo de estructurar módulos, interfaces y datos.

Principios Clave.

Se basa en abstracción, modularidad y encapsulamiento, buscando la independencia funcional y cohesión entre módulos.

Estrategias de Diseño.

Incluye análisis de transformaciones y transacciones, enfocándose en las entradas, salidas y funciones descompuestas del sistema.

Verificación y Documentación.

Se verifica la calidad técnica y coherencia de los modelos, y se documenta detalladamente para asegurar la correcta construcción y mantenimiento del software.

Criterios de Validación de Calidad.

Se refiere a las métricas utilizadas para evaluar y mejorar la calidad del diseño de software, asegurando que los módulos sean independientes y cumplan con una única función.

Acoplamiento.

Clasifica la independencia entre módulos. El objetivo es minimizar el acoplamiento para maximizar la independencia, eliminando relaciones innecesarias, reduciendo el número de relaciones y debilitando la dependencia de las relaciones necesarias.

Cohesión.

Mide la intensidad de asociación funcional dentro de un módulo. Se busca lograr módulos altamente cohesivos, con elementos fuertemente relacionados entre sí, pero no con elementos de otros módulos.

Descomposición (Factoring).

Es la separación de funciones en módulos para reducir tamaño, claridad, evitar duplicación de código y separar la 'administración' del trabajo.

Documentación.

Esencial para la legibilidad y comprensión del diseño, permite detectar errores de consistencia y facilita la perfectibilidad por terceros.

Diseño de Experiencia de Usuario: etapas, actividades, técnicas y herramientas.

Introducción.

El diseño de software ha evolucionado a lo largo del tiempo, incorporando diferentes enfoques y técnicas. La arquitectura de información (AI) y el diseño de interacción

(I.X.D) son dos aspectos clave durante las etapas de requerimientos y diseño, que se combinan en el Diseño de Experiencia de Usuario (UXD). Este enfoque busca integrar ambos procesos, A.I y I.X.D, en las primeras etapas de creación de software, siguiendo los principios del UXD y proponiendo un enfoque metodológico para abordar estos procesos de manera integrada.

Criterios Básicos.

- El artículo aborda el Diseño de Experiencia de Usuario (UXD) a partir de los elementos de la Experiencia de Usuario (UX) de Jesse James Garret.
- Se distinguen dos enfoques principales: el de la web como interfaz de software y como sistema de hipertexto.
- Se describen las etapas de creación de un producto de software: Requerimientos, Diseño, Implementación y Prueba, tanto de forma lineal como iterativa.
- El rol del Diseñador de Experiencia de Usuario se concibe como un mediador entre las necesidades del cliente y las del usuario final.
- Se destaca la flexibilidad y la adaptabilidad necesarias en el diseño de software, ya que las decisiones de diseño varían según el contexto, los contenidos y los usuarios específicos de cada proyecto.

Metodología.

El enfoque metodológico propuesto en el presente trabajo se basa en las etapas del diseño de software descritas anteriormente. Un enfoque que consta de cuatro elementos básicos para su comprensión:

- Etapas: Son las que responden al tiempo transcurrido ¿cuándo?
- Actividades o tareas: Son las que responden al proceso ¿qué?
- Técnicas: Son las que responden a lo que se hace para lograr los procesos y tarea ¿cómo?
- Herramientas: Son las que responden a qué se usa para aplicar las técnicas ¿con qué?

Etapas.

Las etapas se dividen en cuatro momentos por los que transcurre la labor del diseñador del software:

- **Investigación:** en la que se obtiene toda la información posible del proyecto, usuarios y producto a diseñar.
- **Organización:** en la que se procesa toda la información para convertirla en un producto.
- **Diseño:** en la que se plasma el diseño del producto a partir de lo organizado.
- **Prueba:** en la que se comprueba la calidad del diseño propuesto.

Estas etapas pueden ser vistas de la misma manera que se vieron las etapas del ciclo de vida del software, es decir, tanto de forma lineal como iterativa. El resultado de estas etapas se materializa en forma de entregables o artefactos.

Actividades o Tareas.

A continuación, se muestran las actividades o tareas que se realizarían principalmente durante cada etapa:

Etapas 1 - Investigación:

Obtener información esencial sobre el proyecto, clientes y usuarios.

- Definir necesidades generales del proyecto, temática, objetivos del cliente y la intención comunicativa.
- Estudiar a los usuarios: caracterizar, definir perfiles, investigar necesidades y escenarios.
- Analizar el contexto de uso: características culturales, políticas, económicas, sociales y tecnológicas.
- Realizar un modelo de negocio, matriz DAFO, banco de problemas y estudiar productos similares.
- En caso de rediseño, analizar el uso del producto existente y evaluar su calidad.
- Estudiar los contenidos: inventariar recursos, crear mapas de conceptos y contenidos.

- Planificar una estrategia de trabajo y analizar los recursos disponibles.
- Generar informes, organigramas y mapas como artefactos de esta etapa.

ETAPA 2 – ORGANIZACIÓN.

Esta es la etapa que más se relaciona con la expresión artística. Es cuando el diseñador usa tanto criterios científico-técnicos como culturales para organizar toda la información obtenida durante la etapa anterior.

- Representar todas las estructuras posibles de los contenidos, en correspondencia a las necesidades de usuarios y su contexto.
- Definir de todas las formas de jerarquizar las temáticas y los contenidos.
- Hacer corresponder las estructuras planteadas a las necesidades tanto de emisores (clientes) como de receptores (usuarios).
- Definir todos los flujos funcionales que tendrá el producto de software, correspondiendo con los flujos reales de los usuarios en su contexto.

Los artefactos generados durante esta etapa serían: Diagramas manuscritos y anotaciones en forma de borrador.

ETAPA 3 – DISEÑO.

Es la etapa en la que se plasman los resultados de la etapa anterior, ahora con todos los requerimientos técnicos con el fin de que sean comprendidos por usuarios, cliente y el resto del equipo de trabajo.

- Definir la estructura del producto (taxonomía y diagramas de organización o blueprints).
- Definir el funcionamiento del producto (diagramas de funcionamiento).
- Definir las pantallas del producto (diagramas de presentación o wireframes).
- Definir los servicios y funcionalidades que tendrá el producto.
- Definir las etiquetas del producto (labeling).
- Crear prototipos de bajo y alto nivel.

Los artefactos generados durante esta etapa serían: Diagramas digitales y prototipos, informes del producto.

ETAPA 4 – PRUEBA.

Es la etapa en la que se comprueban las propuestas de diseño. Las pruebas se realizan tanto con clientes como con usuarios. Con los clientes con el objetivo de saber si se han logrado los objetivos y demandas planteadas. Y con los usuarios, con el objetivo de saber resueltas las necesidades identificadas.

- Pruebas de prototipos.
- Revisión de diagramas.
- Comprobación de robustez en el etiquetado.
- Comprensión de los servicios diseñados por parte de los usuarios.

Los artefactos generados durante esta etapa serían: Informes evaluativos.

Técnicas.

Las técnicas no están obligatoriamente relacionadas con las actividades ni las etapas, por lo que pueden usarse en cualquier momento del proyecto. Todo depende de lo que se quiera obtener con la técnica. Hay técnicas que tienen mayor uso durante alguna etapa o actividad, pero no por ellos están restringidas a éstas. Como dato curioso, se pueden mezclar técnicas entre sí para lograr algún objetivo específico. Por ejemplo: Aplicar un card sorting a expertos, o realizar una tormenta de ideas dentro de un Diseño Participativo.

Se muestran algunas técnicas usadas según su función principal, aunque no exclusiva:

- Técnicas básicas dentro de un proceso de diseño:
 - Reunión.
 - Entrevista y Encuesta.
 - Observación.
 - Revisión bibliográfica.
 - Consulta a experto.
 - Representación y mapeo.
- Técnicas muy usadas para la búsqueda de información:

- Tormenta de ideas (brainstorming).
- Focus group.
- Crítica de diseños anteriores.
- Diseño participativo.
- Tormenta de idea de necesidades de información (brainstorming).
- Escenarios.
- Personas.
- Benchmarking (análisis competitivo).
- Análisis de frecuencia de texto (repetición de términos y conceptos en los contextos).
- Técnicas muy usadas para la organización:
 - Organización de tarjetas (card sorting).
 - Agrupación (finalizando en un Diagrama de Afinidad).
 - Secuencia (Análisis de secuencia) (Ronda León, Mesa Rábade; 2005).
 - Tabulación de contenidos (crear tablas con los diferentes contenidos a organizar).
 - Validación de términos.
- Técnicas muy usadas para el funcionamiento del producto:
 - Análisis de tareas (Task Analysis).
 - Flujogramas.
 - Paseo cognitivo (Cognitive Walkthrough).
- Técnicas muy usadas para diseñar el producto:
 - Diagramación en papel.
 - Diagrama de organización (Blueprint).
 - Diagrama de funcionamiento (Flow diagram).
 - Diagrama de presentación (Wireframe).
 - Etiquetado (Labeling).
 - Prototipado digital.
- Técnicas usadas para hacer los test del diseño planteado:
 - Crítica de diseño con usuarios y clientes (Critic design).

- Pruebas con usuarios (User test).
- Evaluación heurística (Heuristic test).
- Mapeo de clics dentro de la pantalla del producto (MouseTracking).
- Monitorización o seguimiento visual (EyeTracking).

Herramientas

Las herramientas para el trabajo del diseñador de experiencia de usuario varían según las posibilidades económicas y tecnológicas del proyecto. En el mercado existen tanto herramientas profesionales de un alto costo monetario como herramientas gratuitas.

Aunque existen numerosas aplicaciones que sirven de herramientas para el trabajo del diseñador de experiencia de usuario, el papel y el lápiz han sido y siguen siendo las más usadas. Claro está que los “tables” con sus pantallas “touch” han venido a sustituir en algunos casos al papel y al lápiz, gracias a las aplicaciones que funcionan como tales en estos dispositivos.

Tema 3: Modelo de Datos.

El modelamiento de datos. Modelo entidad relación de Chen. Modelo conceptual de datos. Modelo conceptual canónico. Refinamiento del modelo conceptual de datos. Modelo lógico de datos. Fases cualitativa, cuantitativa y de optimización.

Modelamiento de Datos.

Proceso de definir la estructura lógica y física de los datos en un sistema, incluyendo sucesivos refinamientos para alcanzar calidad.

Características Esenciales.

Se debe aplicar la normalización hasta el nivel más conveniente, manteniendo en mente el objetivo y alcance del sistema. La flexibilidad, mantenibilidad y portabilidad son cruciales.

Factores Críticos.

Trabajar interactivamente con los usuarios, seguir una metodología clara, considerar la estructura e integridad de la información y utilizar diagramas para representar el modelo de datos lógico.

Beneficios.

Un diseño correcto de la base de datos optimiza la tecnología para su desarrollo, anticipa cambios futuros, y facilita la comprensión de los datos por parte de los usuarios sin necesidad de implementar el sistema.

Modelo Entidad - Relación:

Tipos de Entidades.

- Regulares: Existen por sí mismas.
- Débiles: Dependen de otras para existir.

Relaciones y Roles.

- Relaciones: Asociaciones entre entidades.

- Roles: Papel que juega una entidad en una relación.

Atributos y Cardinalidad.

- Atributos: Propiedades de entidades y relaciones.
- Cardinalidad: Número mínimo (el valor que se anota es de cero o uno) y máximo (puede ser uno o muchos) de asociaciones posibles.

Reglas de Negocio.

- Insertar: Condiciones para añadir ocurrencias.
- Borrar: Condiciones para eliminar ocurrencias.

Independencia de Datos.

- Física: Cambios en hardware no afectan el esquema conceptual.
- Lógica: Cambios en el esquema conceptual no afectan las aplicaciones.

Modelos de Datos:

Mundo real.

Contiene la información tal cual la percibimos como seres humanos. Es el punto de partida.

Esquema conceptual.

Representa el modelo de datos de forma independiente del DBMS que se utilizará.

Esquema canónico (o de base de datos).

Representa los datos en un formato más cercano al del ordenador.

Esquema interno.

Representa los datos según el modelo concreto de un sistema gestor de bases de datos (por ejemplo Oracle).

Base de datos física.

Los datos tal cual son almacenados en disco.

Diseño de Datos.

Busca almacenar datos sin redundancia y con fácil acceso.

- Objetivo: generar un modelo de datos que permita almacenar los datos sin innecesaria redundancia y que permita un fácil acceso a los mismos.
- Problemas:
 - Información Repetida.
 - Inhabilidad de Representación.
 - Pérdida de Información.

La mayoría de estos problemas pueden ser resueltos en el modelo lógico de datos (Entidad-Relación).

Modelo Conceptual Canónico.

Define un esquema independiente del DBMS, sin datos redundantes y preservando dependencias.

Normalización (Codd).

- Es un proceso reversible mediante el cual los datos de un modelo de datos se ponen en la forma más simple e independiente.
- Permite que, si la estructura física para los datos que se implemente está normalizada, siempre será posible satisfacer cualquier requerimiento sobre los datos.
- MDC: 1FN; 2FN; 3FN; 4FN; 5FN.
- 1FN: Con cada renglón contiene valores atómicos en cada celda.
- 2FN: se refiere a una relación en una base de datos en la que los atributos no primarios dependen funcionalmente de la clave primaria, y la relación ya está en Primera Forma Normal (1FN). El Número Único de Base (NUB) puede asociarse a un elemento de una matriz de orden n, donde n es el número de entidades que participan en la relación. Cada elemento del NUB tiene tantos padres (e índices en los atributos) como entidades participantes

en la relación. Por ejemplo, Cijk representa la cantidad del producto i programado para ser producido en la máquina j en el período k.

- 3FN: Se aplica a una relación en una base de datos cuando está en Segunda Forma Normal (2FN) y todos sus atributos no primarios no dependen transitivamente de la clave primaria.

Eliminación de redundancia.

Se refiere a la reducción de la duplicación de datos en las entidades. Hay dos tipos de redundancia: estructural y de datos.

- La redundancia estructural ocurre cuando dos entidades tienen atributos idénticos (o en un porcentaje dado), pero no hay una coincidencia de sujetos entre ellas.
- La redundancia de datos ocurre cuando una misma ocurrencia puede tener los mismos datos en ocurrencias diferentes de las entidades. La eliminación de esta redundancia ayuda a mejorar la eficiencia del almacenamiento de datos y a mantener la integridad de la información en la base de datos.

Redundancia en Entidades.

Las entidades son redundantes si tienen los mismos atributos o si una entidad tiene todos sus atributos repetidos en otra. Esto puede llevar a problemas como la información repetida o la pérdida de información.

Redundancia de Relaciones.

Una relación es redundante si la información que proporciona es la misma que otra relación indirecta entre las mismas entidades. Las relaciones redundantes deben eliminarse para evitar duplicaciones innecesarias.

Modelo Conceptual Multidimensional.

Incluye esquemas como el de Estrella y Copo de Nieve, diferenciando entre hechos (valores almacenados) y dimensiones (variables descriptivas).

Hechos y Dimensiones.

Los hechos son valores almacenados en un cubo de datos, mientras que las dimensiones son variables descriptivas que sirven como ejes del cubo.

Cuarta Forma Normal (4FN).

Se alcanza cuando una relación está en 3FN y todas las dependencias de valores múltiples cumplen ciertas condiciones, como ser triviales o que X sea una superllave.

Quinta Forma Normal (5FN).

Una relación está en 5FN si, además de estar en 4FN, cumple con condiciones específicas para todas las dependencias de productos, como ser trivial o que cada R_i sea una superllave.